E5 Completion Guide

This guide summarizes everything implemented for Phase E5, including key files, behaviors, and testing.

1. Execution Models (`src/slice/models/execution.py`)

- **`TradeLeg`**: `asset`, `direction="LONG"`, `size_pct` (0–100).

- **`TradePlan`**: `thesis_id`, `total_notional>0`, `legs`.

- **`ThesisPnL`**: `invested_notional`, `current_value`, `unrealized_pnl`, pct.

- **`SizingConstraints`**: placeholders for future (max leverage, weights, risk).

2. Paper Execution (`src/slice/execution/paper.py`)

- **`PaperExecutionAdapter`**

  - `create_plan_from_thesis()` enforces positive notional, long-only, ⩽100% allocation, mirrors thesis expression.

  - `execute_plan()` buys each leg immediately using `PriceSource` prices, storing `TradeType.SIMULATED` trades with UUID ids.

  - Throws `ValueError` for invalid plan inputs, `RuntimeError` for bad prices.

3. Thesis Repository Update (`src/slice/repositories/thesis_repo.py`)

- Added `update(thesis)` to write changes (primarily status=ACTIVE) without using `insert()`.

4. DataAccess Extensions (`src/slice/intelligence/context/data_access.py`)

- Added `get_thesis_pnl(thesis_id)` to compute invested/curr value, unrealized P&L from stored trades (long-only assumption, uses current prices).

5. Evaluation Session (`src/slice/sessions/thesis_evaluation_session.py`)

- E5 evaluation hook now calls `create_plan_from_thesis(total_notional=100_000.0)` with new naming.

- Stores plan dict in result when adapter succeeds.

6. Approve Plan API (`src/slice/api/session_routes_e4.py`)

- Added `ApprovePlanRequest` (`total_notional` optional, default 100k).

- Added dependency import for `get_price_source_instance` and `PaperExecutionAdapter`.

- New `POST /api/v1/thesis/{thesis_id}/approve-plan` route:

  - 404 if thesis missing.

  - 400 if thesis already ACTIVE or plan invalid (short legs, bad weights, non-positive notional).

  - Executes plan, inserts trades, marks thesis ACTIVE via `thesis_repo.update`.

  - Returns `{status, thesis_id, total_notional, executed_trades}`.

7. Test Coverage (`tests/e5/`)

- **Unit**:

  - `test_plan_generation.py` covers plan creation constraints (sum checks, shorts, notional >0).

  - `test_plan_execution.py` covers trade quantities, zero legs filtered, timestamping, price errors.

  - `test_thesis_pnl.py` covers P&L helper scenarios: simple trade, multiple lots, no trades, multi-asset.

- **Integration**:

  - `test_approve_plan_endpoint.py` covers happy path, double approval rejection, short rejection, missing thesis, price failure, default notional, already ACTIVE.

All tests run via `pytest tests/e5/ -v`, producing 21 passed tests (Deprecation warnings only for `on_event` and `datetime.utcnow()`; functional behavior verified).

How to Use

1. **Creating Plans**: instantiate `PaperExecutionAdapter` with repos/price source, call `create_plan_from_thesis(thesis, total_notional)`.

2. **Executing Plans**: pass resulting plan to `execute_plan(plan, as_of=None)` and trades are recorded immediately.

3. **Approving Plans**: `POST /api/v1/thesis/{id}/approve-plan` with optional `total_notional`. Response includes executed trade details.

4. **P&L**: call `DataAccess.get_thesis_pnl(thesis_id)` for UI-ready stats.

5. **Evaluation Session**: optional plan generation outputs the new DTO for review when adapter is provided.

This setup provides a clean, long-only paper trading path with deterministic plan sizing, execution, and per-thesis performance metrics, ready for future sizing-engine upgrades.